

Aim

To design and develop a secure web application that implements functional security requirements (input validation, command whitelisting, access control, XSS prevention) and nonfunctional security requirements (compression, caching, deployment hardening, performance) using Node.js and Express.js.

Software Required

Software	Purpose
Node.js (v18+)	Server-side JavaScript runtime
Express.js v4.18	Web application framework
compression module	Response compression middleware
VS Code / any editor	Code editor
Web browser (Chrome/Firefox)	Testing and output verification
Vercel (optional)	Serverless cloud deployment

Theory

Functional security requirements define what the system does to enforce security — such as validating user input, restricting allowed operations via whitelisting, preventing path traversal attacks, sanitizing data before sending it to clients, and controlling access to hidden or sensitive files.

Nonfunctional security requirements define how well the system performs its security duties — such as compressing responses to reduce exposure, setting proper cache-control headers, limiting input lengths to prevent abuse, deploying over HTTPS with DDoS protection, and monitoring system health.

Together, they ensure a web application is both functionally safe and operationally resilient.

Additional Learning

1. **Input Validation** — Validate the type, format, and length of all user inputs server-side.
2. **Whitelisting** — Allow only known-safe commands instead of blocking dangerous ones.
3. **Avoid eval() and Shell Execution** — Use safe JavaScript logic to prevent OS-level attacks.
4. **Separation of Concerns** — Enforce security rules on the server, not only the client.
5. **Nonfunctional Security** — Use HTTPS, compression, caching, and monitoring for secure operations.

Conclusion

The application combines functional and nonfunctional security measures to protect against attacks and improve reliability. Features like input validation, command whitelisting, sanitization, XSS protection, TLS, and monitoring help build a secure web application.